表达式的主要作用和副作用（重点）

表达式的主要作用：表达式一定会计算出一个结果，这个结果就是表达式的主要作用。

一般来说，表达式都是存在主要用的。
思考：如何理解表达式的主要作用？（重要）
当小表达式处在大表达式当中时，小表达式的主要作用发挥作用。
在大表达式中，每一个小表达式都是主要作用在起作用。但是副作用也不能忽略。

表达式的副作用：一个表达式有除了主要作用外的作用，就是副作用
常见的副作用有：
1.（最常见的）修改变量的值，赋值。所有带赋值运算符"="的表达式，都具有这个副作用。
int a = 10
2.执行了IO操作。
函数调用就是一个表达式，这时表达式的主要作用是函数的返回值。
比如printf函数，它会返回一个int整数，这是主要作用。副作用是输出数据到终端显示器上。
....
副作用和主要作用是相对独立的，它们之间是不会互相干扰的。

举例：
1.a + b
主要作用： 返回 a+b的值
副作用： 没有副作用

2.a < < 1
主要作用： 返回 a<<1的值。
副作用： 没有

3.a=(a<<1)
主要作用： 返回a<<1的值
副作用： a的值被修改了。 a<<1

4.int c = a + b
a+b : 主要作用： 返回a+b的值； 没有副作用

int c = (a+b)： 主要作用： 返回c的值。副作用： c的值被修改了。

5.while((ch = getchar()) != '\n')

getchar(): 主要作用：返回stdin里面的一个字符； 副作用： 执行了IO操作
ch = getchar()： 主要作用： 返回ch的值； 副作用： 修改了ch的值
(ch = getchar()) != '\n'： 主要作用： 返回ch是否是 \n. 副作用： 没有

6.printf("%d", a++)
a++ ; 主要作用： 返回old a; 副作用： a+1
printf("%d", a++); 主要作用： 打印成功的字符数； 副作用： 执行IO操作。

总结：
1.表达式的主要作用在更大的表达式中发挥作用
2.表达式的副作用相对独立，不受主要作用影响

理解表达式的主要作用和副作用，可以帮助我们更好的理解表达式。更大的表达式。

运算符的优先级和结合性

优先级、 结合性： 主要控制的就是这个表达式的运算顺序。
a+b*c-1+2

首先看优先级。 优先级如果相同，这时候，看结合性，到底是左结合还是右结合。

比如，现在碰到一个表达式。如何去查这个表。
1. a+b*3
a+(b*3)

2. a*b/c

3.如果现在有一个数组下标， [a,b]。 怎么求它的中间下标。
(a+b)/2: 容易越界。 如果a和b都接近最大值。

(b - a) / 2 + a: 推荐大家这样写。
(b-a>>1) + a: 后续，进公司了，有人这么写，要能认识。 但是自己写的时候， 按照第二种写法写。
((b-a) >> 1) + a: -的优先级比 >> 高

4.结构体。 就是一个类型，里面能装其他的数据。
p.age++;

(p.age)++;

5. a = b = c = 10;

(a=(b= (c=10)))

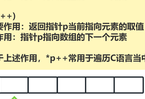
在C里面，我们聊的内存都是虚拟内存。
&num *解引用， 就是拿出对应地址的值。
假设p是一个指针， 就是存储地址的变量。

*p++:后续会经常用到。
*(p++)
优先级： 后缀++优先级更高， *的优先级低

p++: 主要作用：返回p原本的指向。 副作用： p+1

*(p++) : 主要作用： 返回p地址指向的值；

整体： 返回p原本指向的值； 将p指针往后挪1



运算符的优先级：多个运算符组成表达式时，哪一个运算符先计算，这就是优先级
运算符的结合性：多个相同优先级的运算符组成表达式时，结合性决定了从左往右还是从右往左计算。也就是左结合性，和右结合性。

优先级的特点总结（重点）：
1.一元运算符的优先级总是高于二元运算符。
2.算术运算符的优先级仅次于单目运算符，是二元运算符中优先级最高的。
3.赋值运算符(包括复合赋值运算符)的优先级几乎是最低的。这很好理解，如果运算还没结束，赋值就完成了，这不是想看到的。
4.当不确定优先级或者为了代码清晰时，使用括号来明确运算顺序。这不仅可以避免优先级引起的错误，还可以使代码更易于理解。

举例：
1.求数组区间中间索引的惯用法(优先级的案例)
求数组[a, b]区间的中间索引，怎么求呢？
对于C/C++来说，(a + b) / 2存在溢出的可能性，所以尽量不要这么用
(b - a) / 2 + a
优化一下, 用右移运算符优化
((b - a) >> 1) + a 正确的
(b - a) >> 1 + a +先计算, 错误
b - a >> 1 + a +先计算, 错误
(b - a >> 1) + a 正确的

2.结合性的例子
结构体类型
比如存在一个Student类型的结构体, 它具有age成员, 表示学生的年龄
s1是一个Student结构体的对象/实例
对于表达式:
(s1.age)++;
.和++的优先级是一样的, 所以考虑结合性, 是左结合性的, 所以从左往右计算

再比如指针p是一个指向s1结构体对象的指针, 于是就存在下列的表达式:
(p->age)++;
->和++的优先级是一样的, 所以考虑结合性, 是左结合性的, 所以从左往右计算

3.结合性的例子:
a = b = c = 10;
(a = (b = (c = 10)));
赋值号是右结合性的, 所以最终三个变量的值都是10

自增运算符和自减运算符, 分为前缀和后缀形式. 后缀形式优先级更高
我们可以通过分析表达式的主要作用和副作用来理解, 它们构成的表达式

前缀自增自减:
++a
主要作用: 返回a自增1以后的结果（前缀形式就是先自增自减，后返回结果）
副作用: 改变a变量的值, 使它自增1

后缀自增自减:
a--
主要作用: 直接返回a的结果（后缀形式就是先返回结果，再自增自减）
副作用: 改变a变量的值, 使它自减1

举例:
指针操作当中特别重要，特别常用，一定要理解学会的一个惯用法：
假设指针p是一个指向某个int数组首元素的指针，假如该数组是：
[0, 31, 28, 31.....]
*p++
*称之为解引用运算符，它表示获取指针指向位置元素的取值。
后缀++的优先级比解引用运算符*更高，所以后缀++先计算

p++先计算
主要作用： 返回指针p当前指向的元素的指针
副作用： 指针p指向数组的下一个元素

*(p++)
主要作用： 返回指针p当前指向元素的取值
副作用： 指针p指向数组的下一个元素

基于上述作用，*p++常用于遍历C语言当中的数组以及字符串（尤其是字符串）。

按位运算符中，比较需要留意的是按位异或。它具有以下一些非常优秀的性质：
a ^ 0 = a:任何整数异或0得到的都是它本身
a ^ a = 0:任意整数异或自己得到的都是0
a ^ b = b ^ a:异或运算满足交换律
(a ^ b) ^ c = a ^ (b ^ c): 异或运算满足结合律

C语言的语句语法，需要注意的几个细节：
1.复合语句。
if/for/while等结构，从语法上来说，它们都只能控制紧跟此结构的下一条语句
若希望它们能控制多条语句，必须将多条语句使用{}括起来，这就是复合语句。

2.（重点）
对于if/for/while等结构，即便它们控制的语句就只有一条，那么也不应当省略{}，应当保留{}
这是一个代码规范的问题，因为省略{}容易引发bug

3.三目运算符和if多分支的选择问题：
能用三目运算符就不要用if多分支，但是大多数情况用不了三目运算符。

4.C/C++的switch中，每一个case分支都必须写常量或者宏常量，这种在编译时期能够确定取值的数据。
不能是变量或者需要运行时期确定的数据。
switch这种设计的最大优点是高效率，我们把这种设计方式叫做"硬编码"，在实现某些树形结构时，使用switch比传统的方式要高。

能用switch就用switch，但是能用的机会不多。
switch只适用于对固态的，离散的常量数值做选择

在写if多分支时, 我们尽量把能够给出返回值的选择放在代码的上面进行处理, 这样做利用return结束函数可以减少if的分支数量, 提高代码可读性. 最典型的例子就是函数的参数校验, 校验它不符合规则的情况, 然后直接return

do..while循环
很少用，它的最大特点就是至少也会执行一次。而for、while完全可能一次都不执行

for循环 vs while循环（重点）
for和while循环是完全等价的，能用for一定能用while，能用while一定也能用for

while循环更适用于：
循环次数不能直接确定，或者只知道循环结束条件的情况
比如：
用循环清空stdin缓冲区
用指针来遍历访问数据结构时（字符串，链表，树.....）普遍都会选择while循环

for循环更适用于：
循环次数能够直接确定，或者通过某个变量直观的影响循环的执行时
最典型的就是数组的遍历，因为数组有长度和下标，使用for循环遍历是最合适的